Логістична регресія та SVM 🎯

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay, RocCurveDi
from sklearn.preprocessing import StandardScaler
```

In [2]:
```
X, y = make_classification(n_samples=500, n_features=2, n_redundant=0,
                           n_clusters_per_class=1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

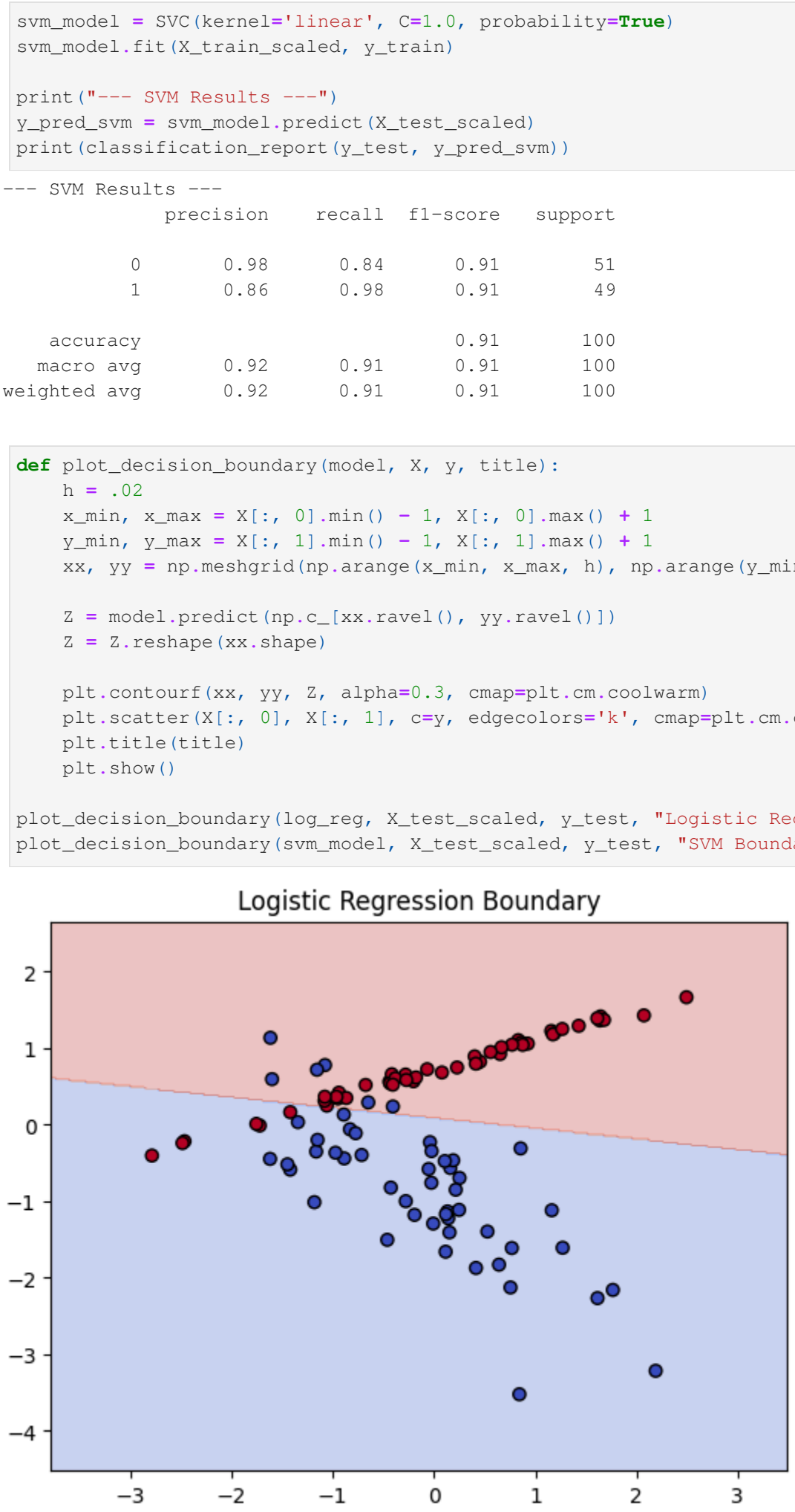
In [3]:
```
log_reg = LogisticRegression()
log_reg.fit(X_train_scaled, y_train)

print("--- Logistic Regression Results ---")
y_pred_log = log_reg.predict(X_test_scaled)
print(classification_report(y_test, y_pred_log))
```

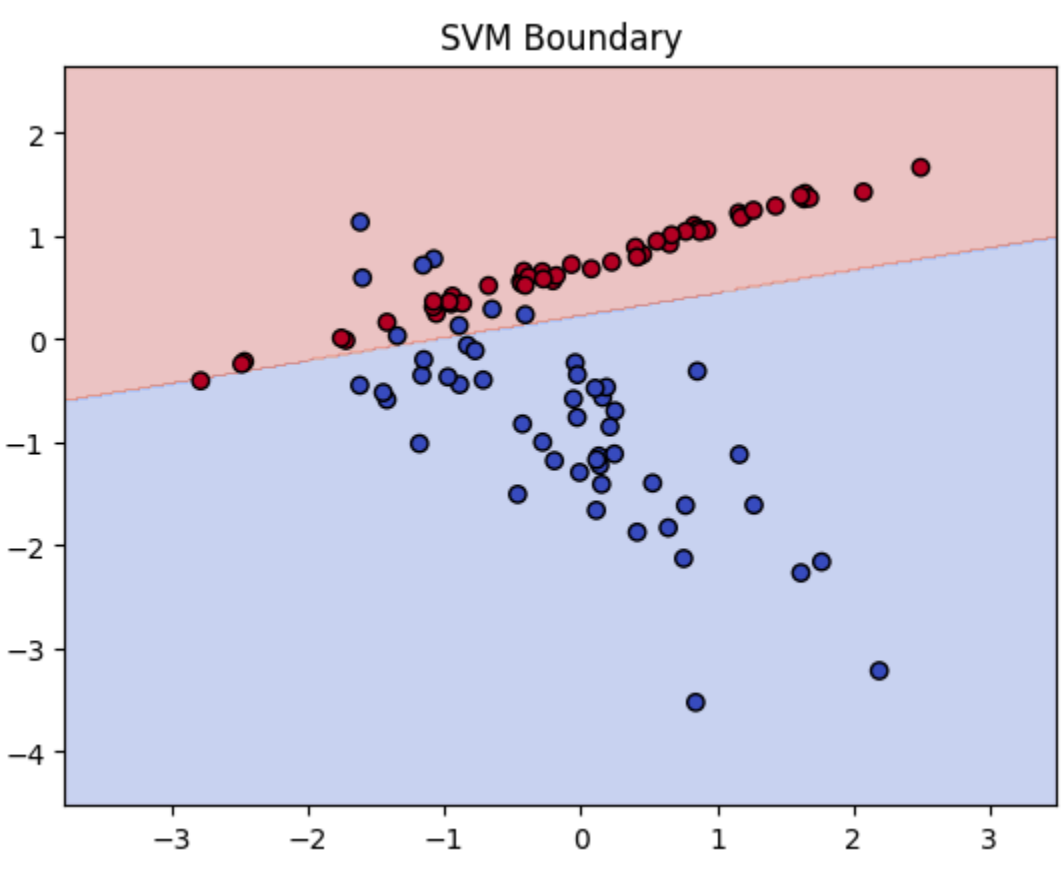
```
--- Logistic Regression Results ---
              precision    recall  f1-score   support

           0       0.88      0.88      0.88        51
           1       0.88      0.88      0.88        49

    accuracy                           0.88       100
   macro avg       0.88      0.88      0.88       100
weighted avg       0.88      0.88      0.88       100
```

In [4]:
```
svm_model = SVC(kernel='linear', C=1.0, probability=True)
svm_model.fit(X_train_scaled, y_train)

print("--- SVM Results ---")
y_pred_svm = svm_model.predict(X_test_scaled)
print(classification_report(y_test, y_pred_svm))
```

```
--- SVM Results ---
              precision    recall  f1-score   support

           0       0.98      0.84      0.91        51
           1       0.86      0.98      0.91        49

    accuracy                           0.91       100
   macro avg       0.92      0.91      0.91       100
weighted avg       0.92      0.91      0.91       100
```

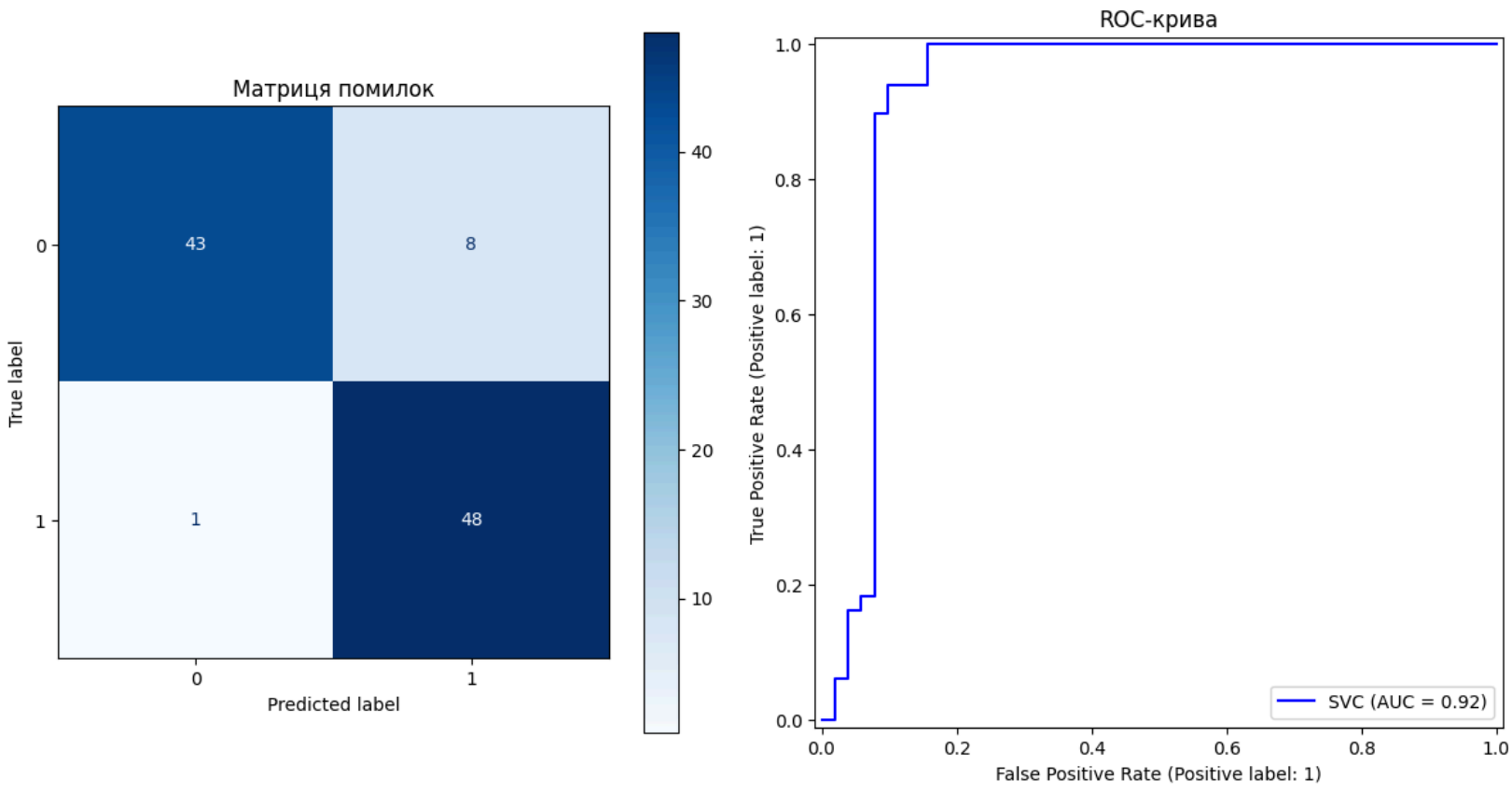
In [5]:
```
def plot_decision_boundary(model, X, y, title):
    h = .02
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.coolwarm)
    plt.title(title)
    plt.show()

plot_decision_boundary(log_reg, X_test_scaled, y_test, "Logistic Regression Boundary")
plot_decision_boundary(svm_model, X_test_scaled, y_test, "SVM Boundary")
```





In [6]:
```
print("--- Візуалізація помилок (Confusion Matrix) ---")

fig, axes = plt.subplots(1, 2, figsize=(12, 6))

ConfusionMatrixDisplay.from_estimator(svm_model, X_test_scaled, y_test, cmap='Blues', ax=axes[0])
axes[0].set_title("Матриця помилок")

RocCurveDisplay.from_estimator(svm_model, X_test_scaled, y_test,  curve_kwargs={'color': 'blue'}, ax=ax
axes[1].set_title("ROC-крива")

plt.tight_layout()
plt.show()
```

```
--- Візуалізація помилок (Confusion Matrix) ---
```


Матриця помилок
True label
Predicted label
0
1
43
8
1
48
40
30
20
10
ROC-крива
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
SVC (AUC = 0.92)